

1. O que é o Event Loop e qual sua importância no Node.js?

O **Event Loop** é o mecanismo responsável por coordenar a execução de operações assíncronas no Node.js. Ele monitora a fila de eventos e executa callbacks quando a pilha de execução (Call Stack) está livre.

Sua importância está em permitir que uma única thread atenda diversas requisições simultaneamente sem bloquear a aplicação, tornando o Node.js altamente eficiente para operações de entrada e saída (I/O), como acesso a arquivos, banco de dados e requisições HTTP.

2. Diferencie CommonJS de ES Modules.

CommonJS	ES Modules
Utiliza <code>require()</code> para importar módulos	Utiliza <code>import</code>
Utiliza <code>module.exports</code> para exportar	Utiliza <code>export</code>
Carregamento síncrono	Carregamento estático
Padrão tradicional do Node.js	Padrão oficial do JavaScript moderno
Ideal para projetos legados	Recomendado para novos projetos

Exemplo CommonJS

```
const soma = require("./soma");  
module.exports = soma;
```

Exemplo ES Modules

```
import { soma } from "./soma.js";  
export function soma(a, b) {  
  return a + b;  
}
```

3. O que são Streams e Buffers?

Streams são fluxos de dados que permitem processar informações em pequenos blocos (*chunks*), sem carregar todo o conteúdo na memória.

Buffers são estruturas utilizadas para armazenar temporariamente dados binários durante esse processamento.

Enquanto os Streams controlam o fluxo dos dados, os Buffers armazenam esses dados temporariamente.

4. Liste três tipos de Streams e suas aplicações.

Readable Stream

Utilizada para leitura de arquivos, vídeos, áudios e dados da rede.

Writable Stream

Utilizada para gravação de arquivos, respostas HTTP e geração de logs.

Transform Stream

Permite modificar os dados enquanto eles estão sendo processados, como compressão, criptografia ou conversão de texto.

5. Explique a diferença entre erros de runtime e erros de sintaxe.

Erro de Sintaxe

Acontece quando o código foi escrito incorretamente e nem chega a executar.

Exemplo:

```
console.log("Olá"
```

Erro de Runtime

O código inicia normalmente, porém ocorre um erro durante sua execução.

Exemplo:

```
const usuario = null;  
console.log(usuario.nome);
```

6. Quais são as vantagens de usar Streams para processamento de dados?

As principais vantagens são:

- menor consumo de memória;
- processamento de arquivos grandes;
- maior desempenho;
- processamento contínuo dos dados;
- ideal para uploads, downloads e streaming de vídeos.

7. Como o módulo fs pode ser usado para manipular arquivos?

O módulo **fs (File System)** permite:

- criar arquivos;
- ler arquivos;
- editar arquivos;
- excluir arquivos;
- criar diretórios;
- remover diretórios.

Ele possui versões síncronas (`readFileSync`) e assíncronas (`readFile`), sendo a versão assíncrona a mais recomendada.

8. O que acontece quando um erro não é tratado em um Stream?

Caso um erro não seja tratado pelo evento `error`, o processo pode ser encerrado inesperadamente, causando a interrupção da aplicação.

Por isso é recomendado sempre utilizar:

```
stream.on("error", (err) => {  
  console.error(err);  
});
```

9. Qual é a função do módulo http no Node.js?

O módulo **http** permite criar servidores HTTP sem utilizar frameworks externos.

Com ele é possível:

- receber requisições;
- enviar respostas;
- criar APIs simples;
- configurar rotas;
- manipular cabeçalhos HTTP.

10. Explique o papel do Thread Pool no Node.js.

Embora o JavaScript execute em uma única thread, operações pesadas (como leitura de arquivos, criptografia e compressão) são enviadas para o **Thread Pool**, gerenciado pela biblioteca **libuv**.

Isso evita bloquear o Event Loop, permitindo que a aplicação continue respondendo a novas requisições enquanto essas tarefas são processadas em segundo plano.